8) Queens using Min-Conflicts:

queens = [1, 3, 4, 2]

```
def conflicts(col, row):
    count = 0

    for other in range(4):
        if other == col:
            continue

        if queens[other] == row:
            count += 1
        elif abs(queens[other] - row) == abs(other - col):
            count += 1

    return count

def show_board():
    for r in range(1, 5):
        print(" ".join(
            "Q" if queens[c] == r else "."
            for c in range(4)
        ))
    print("Rows:", queens)

def total_conflicts():
    count = 0

    for i in range(4):
        for j in range(i + 1, 4):
            if (queens[i] == queens[j] or
                abs(queens[i] - queens[j]) == j - i):
                count += 1

    return count

print("Initial board:")
show_board()

while total_conflicts() > 0:
    col = next(
        c for c in range(4)
```

```
        if conflicts(c, queens[c]) > 0
    )

    choices = [
        (conflicts(col, row), row)
        for row in range(1, 5)
    ]

    best_count = min(x[0] for x in choices)
    best_row = min(
        row for count, row in choices
        if count == best_count
    )

    queens[col] = best_row

    print("\nAfter moving column", col + 1)
    show_board()
    print("Total conflicts:", total_conflicts())

print("\nSolution found!")
```